PYTHON ACTIVITY-BASED LEARNING

Selected 10 Case Studies

INTRODUCTION

This document contains ten selected Python case studies based on the supplied activity-based learning list. Each case study includes the problem statement, objectives, assumptions, algorithm, proper step-by-step pseudocode, a vertical flowchart, Python source code, test cases, sample output, and conclusion.

The programs practise variables, data types, operators, conditional statements, loops, lists, nested lists, tuples, and tuple unpacking.

CASE STUDY 1 – UNIVERSITY SCHOLARSHIP ALLOCATION

1. Problem Statement

Design a program to check applicants using average marks, attendance, and family income.

2. Objectives

1. Calculate average marks.
2. Check attendance and income requirements.
3. Separate eligible and ineligible applicants.

3. Assumptions

• Minimum average: 75%.

• Minimum attendance: 75%.

• Maximum annual family income: ■2,50,000.

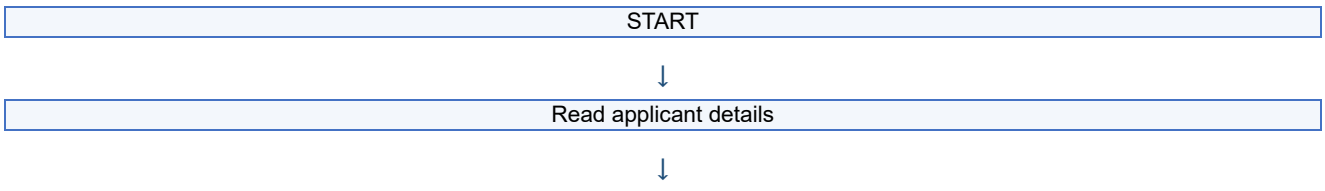
4. Algorithm

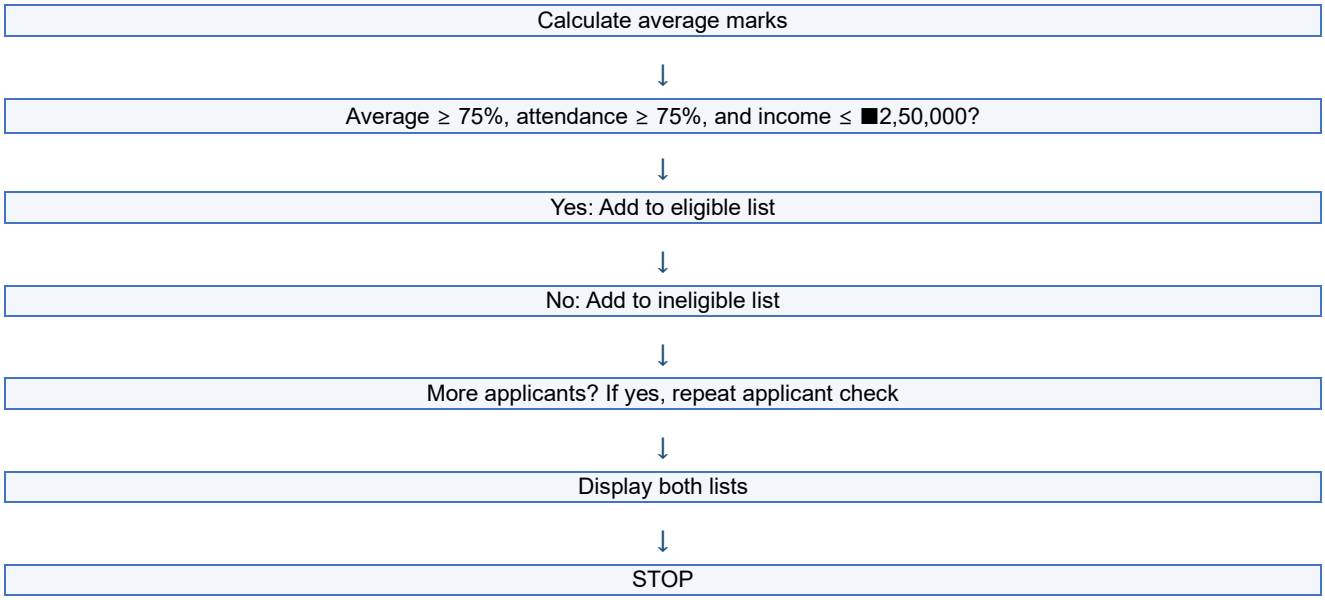
1. Read each applicant's marks, attendance, and income.
2. Calculate the average of three subject marks.
3. Check all scholarship conditions.
4. Store the applicant in the eligible or ineligible list.
5. Display both lists.

5. Pseudocode

```
START
    SET eligible = empty list
    SET ineligible = empty list
    FOR each applicant
        CALCULATE average = (mark1 + mark2 + mark3) / 3
        IF average >= 75 AND attendance >= 75 AND income <= 250000 THEN
            ADD applicant to eligible
        ELSE
            ADD applicant to ineligible
        END IF
    END FOR
    DISPLAY eligible and ineligible
STOP
```

6. Vertical Flowchart





7. Python Source Code

```
applicants = [     ("Anil", 80, 75,
85, 90, 200000),
    ("Bhavya", 70, 80, 75, 85, 180000),
    ("Charan", 90, 88, 92, 80, 300000),
    ("Divya", 78, 76, 80, 70, 220000)
] eligible = [] ineligible = []

for name, m1, m2, m3, attendance, income in applicants:
    average = (m1 + m2 + m3) / 3     if average >= 75 and
attendance >= 75 and income <= 250000:
        eligible.append((name, round(average, 2)))
else:         ineligible.append((name, round(average,
2)))

print("Eligible Applicants:", eligible) print("Ineligible
Applicants:", ineligible)
```

8. Test Cases

Test Case	Input	Expected Result
1	Anil: marks 80,75,85; attendance 90; income 200000	Eligible
2	Bhavya: marks 70,80,75; attendance 85; income 180000	Eligible
3	Charan: marks 90,88,92; attendance 80; income 300000	Ineligible
4	Divya: marks 78,76,80; attendance 70; income 220000	Ineligible

9. Sample Output

```
Eligible Applicants: [('Anil', 80.0), ('Bhavya', 75.0)]
Ineligible Applicants: [('Charan', 90.0), ('Divya', 78.0)]
```

10. Conclusion

The program applies scholarship conditions and separates applicants using lists, loops, operators, and conditional statements.

CASE STUDY 2 – PROGRESSIVE ELECTRICITY BILLING

1. Problem Statement

Calculate an electricity bill using progressive consumption slabs and a fixed charge.

2. Objectives

1. Accept units consumed.
2. Calculate charges for each slab.
3. Display the total bill.

3. Assumptions

• First 100 units: ■2 per unit.

• Next 100 units: ■3 per unit.

• Units above 200: ■5 per unit.

• Fixed charge: ■50.

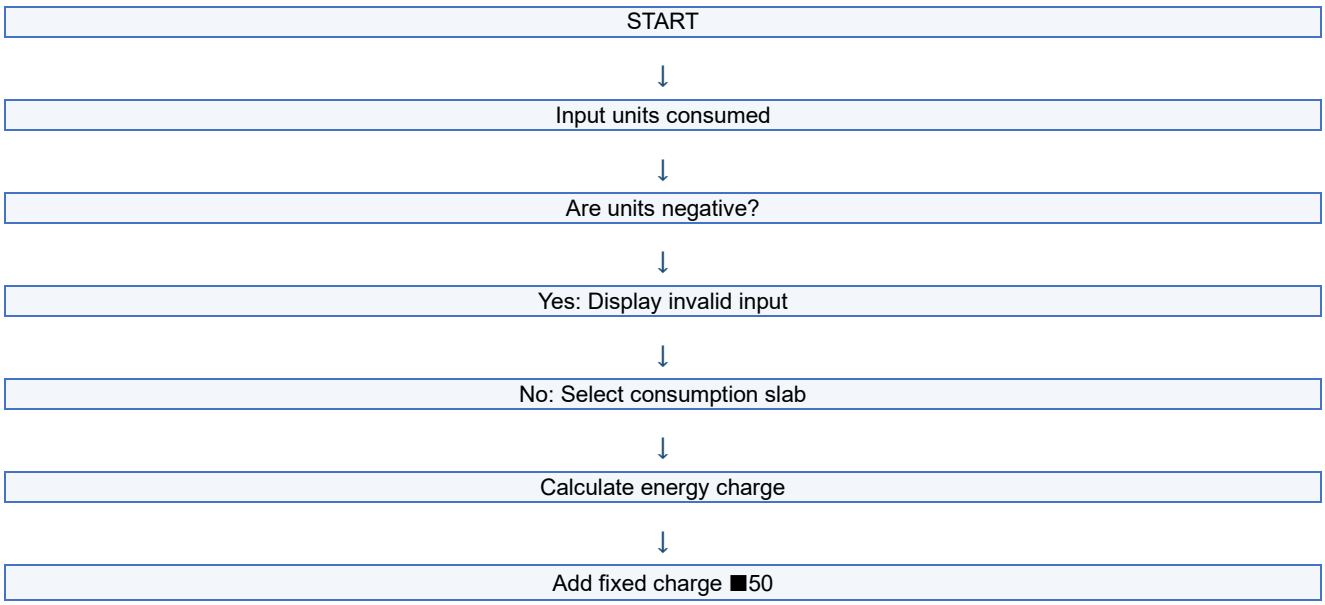
4. Algorithm

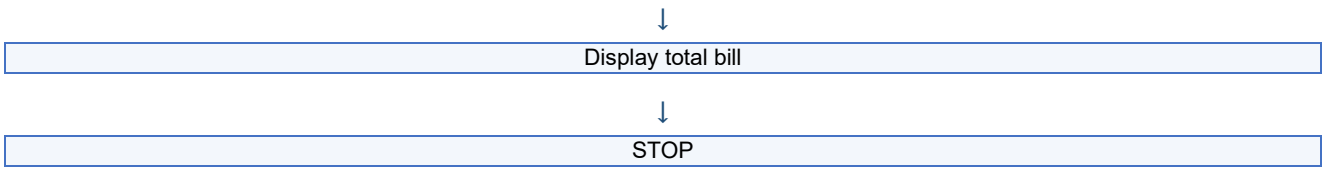
1. Read the units consumed.
2. Calculate the energy charge using the correct slab.
3. Add the fixed charge.
4. Display the bill.

5. Pseudocode

```
START
    INPUT units
    IF units < 0 THEN
        DISPLAY "Invalid units"
    ELSE          IF units <= 100
THEN              charge = units *
2          ELSE IF units <= 200
THEN
            charge = 100 * 2 + (units - 100) * 3
ELSE
            charge = 100 * 2 + 100 * 3 + (units - 200) * 5
END IF
        total_bill = charge + 50
DISPLAY charge and total_bill
    END IF
STOP
```

6. Vertical Flowchart





7. Python Source Code

```
units = int(input("Enter units consumed: "))

if units < 0:
    print("Invalid units")
else:     if units <= 100:
        charge = units * 2
elif units <= 200:
        charge = 100 * 2 + (units - 100) * 3     else:
charge = 100 * 2 + 100 * 3 + (units - 200) * 5

    total_bill = charge + 50
print("Energy Charge: Rs.", charge)
print("Total Bill: Rs.", total_bill)
```

8. Test Cases

Test Case	Input	Expected Result
1	50 units	■150
2	100 units	■250
3	150 units	■400
4	250 units	■800

9. Sample Output

```
Enter units consumed: 150
Energy Charge: Rs. 350
Total Bill: Rs. 400
```

10. Conclusion

The program uses if-elif-else statements to calculate charges under progressive tariff slabs.

CASE STUDY 3 – EMPLOYEE PAYROLL AND DEDUCTION ANALYSIS

1. Problem Statement

Calculate gross salary from basic pay, allowances, and overtime, then subtract deductions to find net salary.

2. Objectives

1. Calculate HRA, DA, and overtime.
2. Calculate gross salary and deduction.
3. Display net salary.

3. Assumptions

• HRA is 20% of basic pay.

• DA is 10% of basic pay.

• Overtime pay is ■200 per hour.

• Deduction is 8% of basic pay.

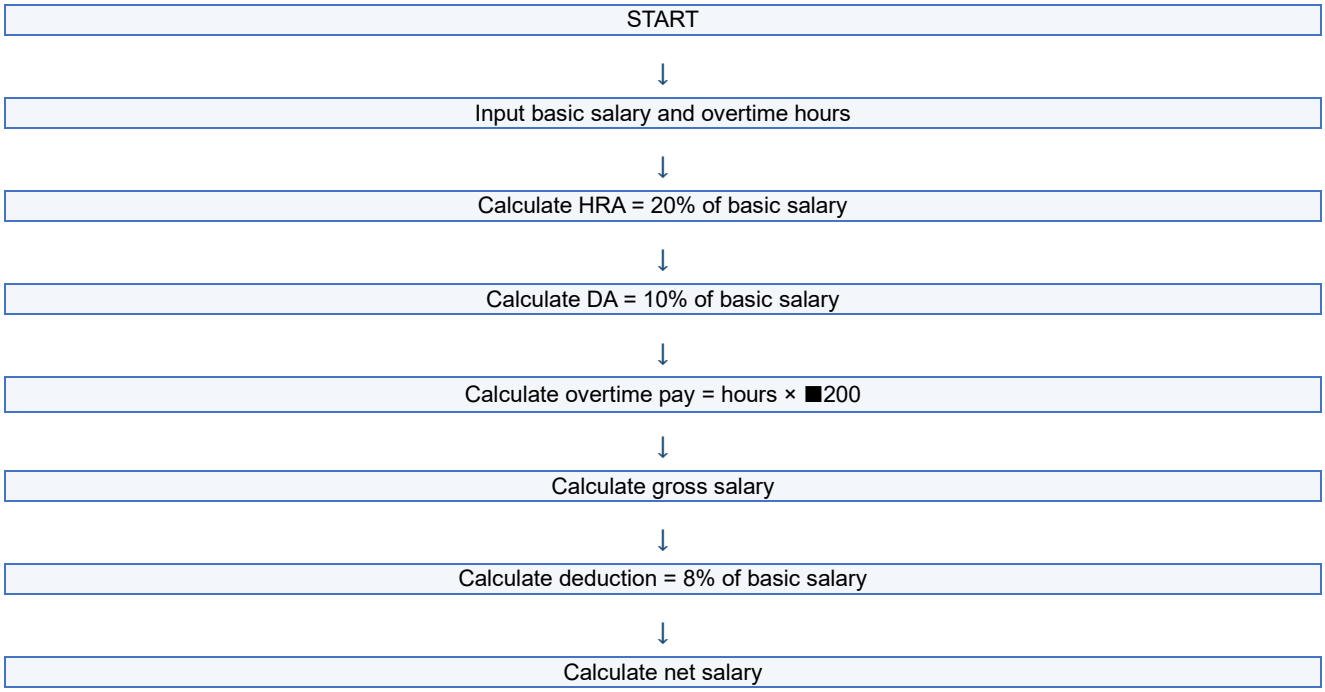
4. Algorithm

1. Read basic salary and overtime hours.
2. Calculate allowances and overtime pay.
3. Calculate gross salary and deduction.
4. Calculate net salary.
5. Display the salary statement.

5. Pseudocode

```
START
    INPUT basic_salary, overtime_hours
    HRA = basic_salary * 0.20     DA = basic_salary * 0.10
overtime_pay = overtime_hours * 200     gross_salary =
basic_salary + HRA + DA + overtime_pay     deduction =
basic_salary * 0.08     net_salary = gross_salary -
deduction     DISPLAY HRA, DA, overtime_pay
    DISPLAY gross_salary, deduction, net_salary
STOP
```

6. Vertical Flowchart



↓

Display salary statement

↓

STOP

7. Python Source Code

```
basic = float(input("Enter basic salary: ")) hours
= int(input("Enter overtime hours: "))

hra = basic * 0.20 da = basic * 0.10
overtime = hours * 200 gross = basic
+ hra + da + overtime deduction =
basic * 0.08 net = gross - deduction

print("HRA:", hra) print("DA:",
da) print("Overtime Pay:",
overtime) print("Gross Salary:",
gross) print("Deduction:",
deduction) print("Net Salary:",
net)
```

8. Test Cases

Test Case	Input	Expected Result
1	Basic ■20,000; OT 5 hours	Net salary ■25,400
2	Basic ■30,000; OT 0 hours	Net salary ■35,600

9. Sample Output

```
Enter basic salary: 20000
Enter overtime hours: 5
HRA: 4000.0
DA: 2000.0
Overtime Pay: 1000
Gross Salary: 27000.0
Deduction: 1600.0
Net Salary: 25400.0
```

10. Conclusion

The program demonstrates arithmetic operations and variables to prepare a simple salary statement.

CASE STUDY 4 – RETAIL BILLING WITH DISCOUNT RULES

1. Problem Statement

Calculate a bill for multiple products, apply a discount above a threshold, and add tax.

2. Objectives

1. Calculate product amounts.
2. Apply the discount rule.
3. Calculate tax and final bill.

3. Assumptions

• 10% discount if subtotal is at least ■1,000.

• Tax is 5% after discount.

• The sample has three products.

4. Algorithm

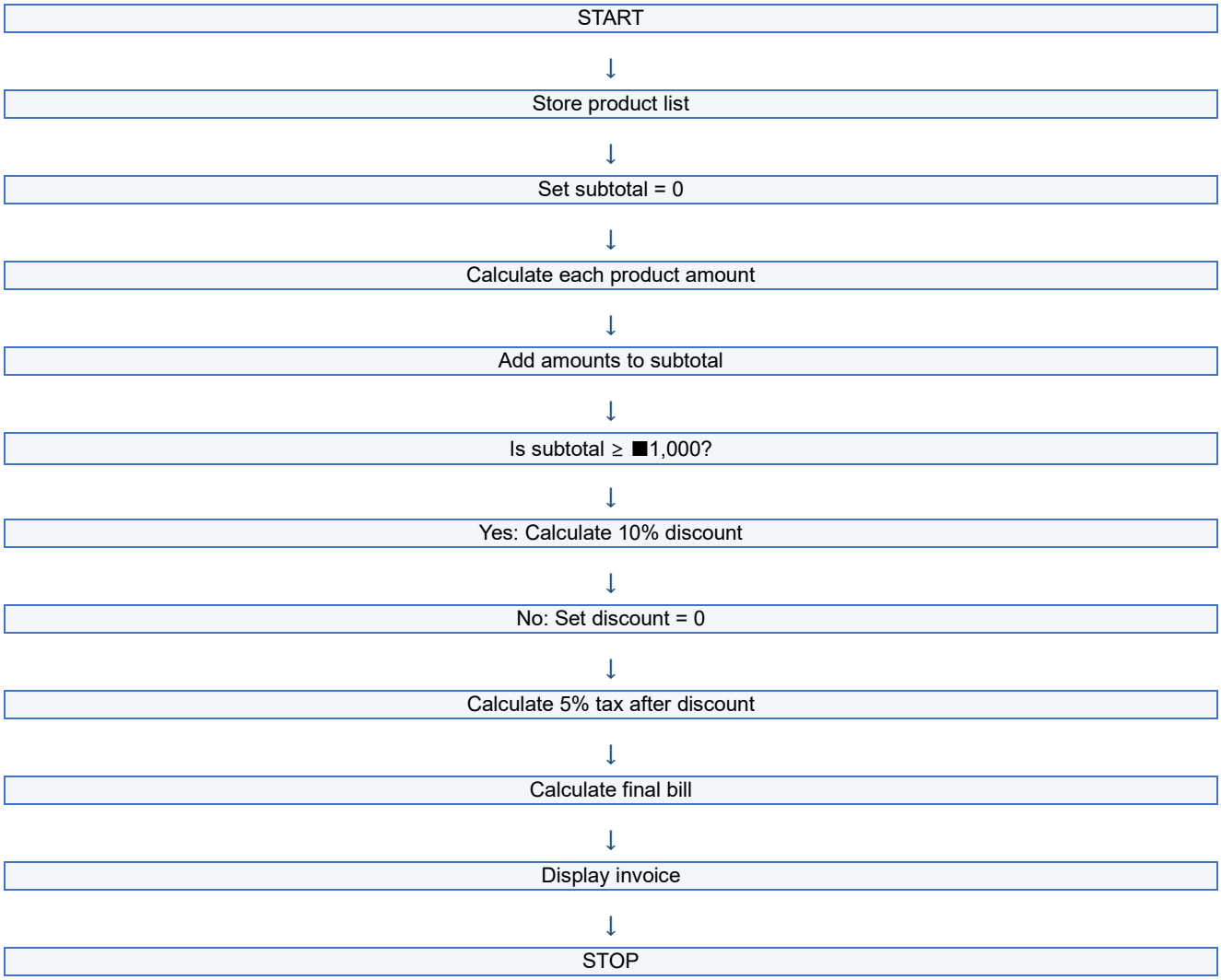
1. Store product names, prices, and quantities.
2. Calculate each product amount and subtotal.
3. Apply the discount condition.
4. Calculate tax and final amount.

5. Display the invoice.

5. Pseudocode

```
START
    SET subtotal = 0     FOR each product
amount = price * quantity         subtotal =
subtotal + amount     END FOR     IF subtotal >=
1000 THEN         discount = subtotal * 0.10
ELSE         discount = 0     END IF
taxable_amount = subtotal - discount     tax =
taxable_amount * 0.05     final_bill =
taxable_amount + tax     DISPLAY subtotal,
discount, tax, final_bill
STOP
```

6. Vertical Flowchart



7. Python Source Code

```
products = [
 ("Notebook", 80, 5),
     ("Pen", 20, 10),
     ("Bag", 700, 1)
 ] subtotal = 0

 for name, price, quantity in products:
     amount = price * quantity
 subtotal += amount
 print(name, ":", amount)

 if subtotal >= 1000:
     discount = subtotal * 0.10
 else:     discount = 0

 tax = (subtotal - discount) * 0.05 total
 = subtotal - discount + tax
```

```
print("Subtotal:", subtotal)
print("Discount:", discount)
print("Tax:", tax)
print("Final Bill:", total)
```

8. Test Cases

Test Case	Input	Expected Result
1	Subtotal ■500	No discount
2	Subtotal ■1,200	10% discount; tax applied after discount
3	Notebook, pen, bag (subtotal ■1,300)	Final bill ■1,228.50

9. Sample Output

```
Notebook : 400
Pen : 200
Bag : 700
Subtotal: 1300
Discount: 130.0
Tax: 58.5
Final Bill: 1228.5
```

10. Conclusion

The program uses a product list, loop, conditional discount, and arithmetic to create an invoice.

CASE STUDY 5 – EXAMINATION RESULT AND GRACE-MARK VALIDATION

1. Problem Statement

Evaluate marks in three subjects using a minimum pass mark and calculate total and average.

2. Objectives

1. Read marks in three subjects.
2. Calculate total and average.
3. Determine pass or fail status.

3. Assumptions

• Three subjects, each out of 100.

• Minimum pass mark is 35 in each subject.

• No grace marks are applied.

4. Algorithm

1. Read marks for all three subjects.
2. Calculate total and average.
3. Check whether every mark is at least 35.
4. Display the result.

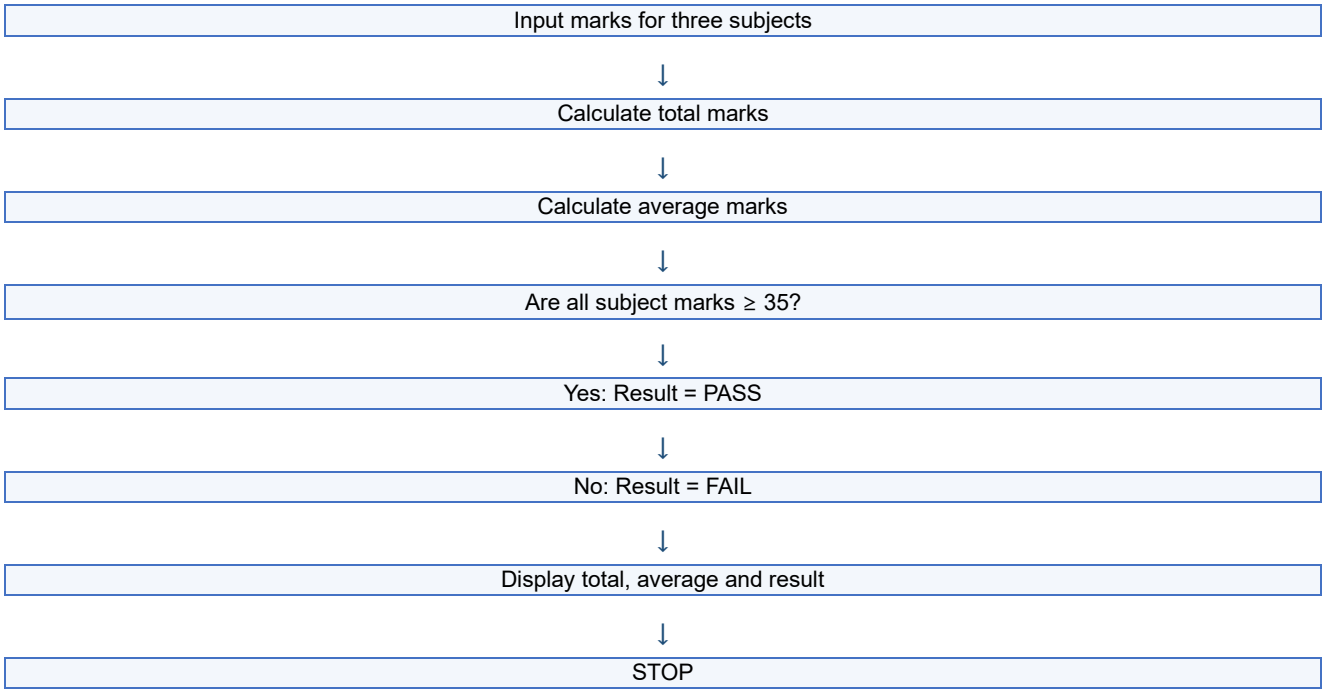
5. Pseudocode

```
START     INPUT mark1, mark2, mark3     total = mark1 +
mark2 + mark3     average = total / 3     IF mark1 >= 35
AND mark2 >= 35 AND mark3 >= 35 THEN         result =
"PASS"     ELSE
        result = "FAIL"
END IF
    DISPLAY total, average, result
STOP
```

6. Vertical Flowchart



↓



7. Python Source Code

```
marks = []
for i in range(3):     mark = int(input("Enter
subject mark: "))     marks.append(mark)

total = sum(marks)
average = total / 3

if all(mark >= 35 for mark in marks):
    result = "PASS"
else:     result =
"FAIL"

print("Marks:", marks) print("Total:",
total) print("Average:",
round(average, 2)) print("Result:",
result)
```

8. Test Cases

Test Case	Input	Expected Result
1	70, 80, 90	PASS; total 240; average 80
2	35, 40, 60	PASS; all subjects meet pass mark
3	80, 30, 75	FAIL; one subject below 35

9. Sample Output

```
Enter subject mark: 70
Enter subject mark: 80
Enter subject mark: 90
Marks: [70, 80, 90]
Total: 240
Average: 80.0
Result: PASS
```

10. Conclusion

The program uses a list, loop, arithmetic, and conditions to evaluate student results.

CASE STUDY 6 – ATTENDANCE RECOVERY COMPUTATION

1. Problem Statement

Calculate current attendance and the minimum future classes that must be attended to reach 75%.

2. Objectives

1. Calculate current attendance percentage.
2. Use a loop to find required future classes.
3. Display the result.

3. Assumptions

• Attendance threshold is 75%.

• The student attends every future class.

• Total classes increase by one for every future class.

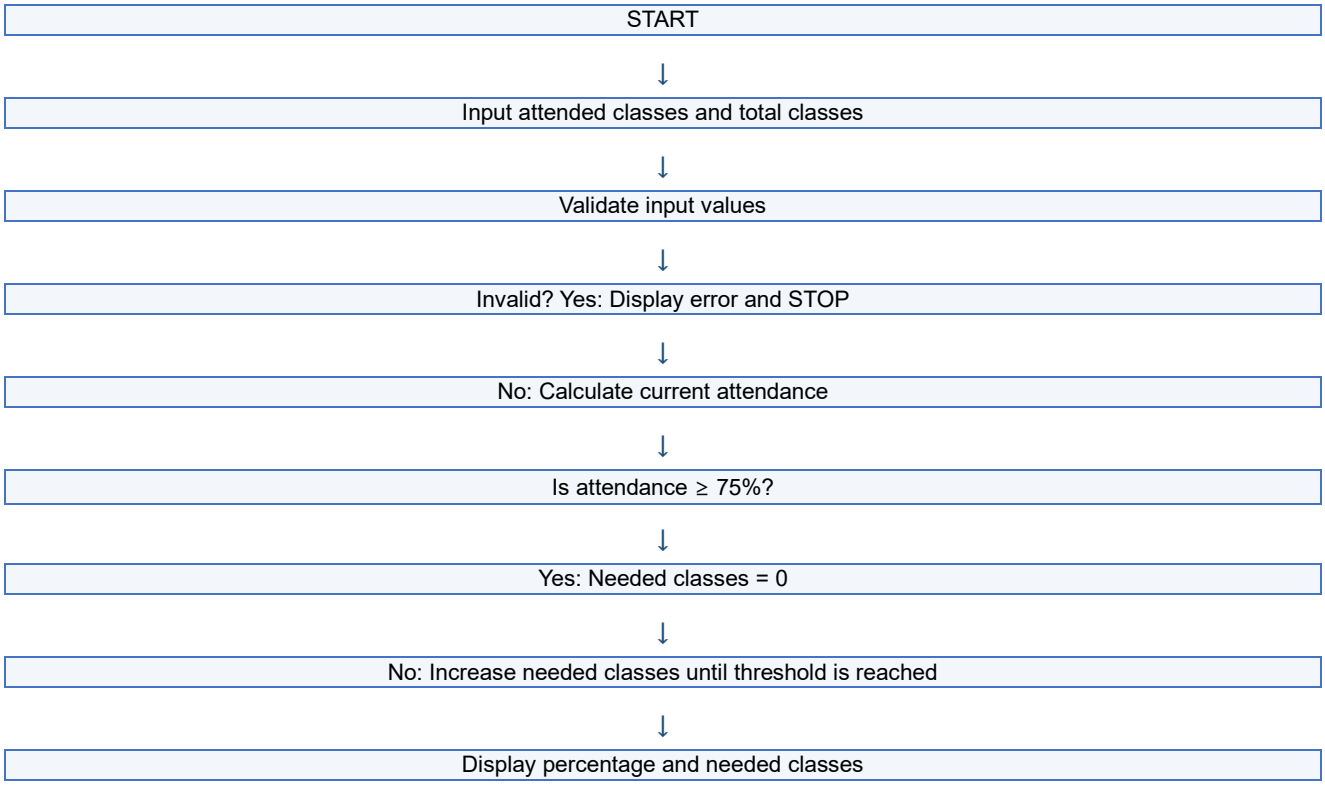
4. Algorithm

1. Read attended and total classes.
2. Validate total classes and attendance counts.
3. Calculate current percentage.
4. Increase the future class count until attendance reaches 75%.
5. Display the answer.

5. Pseudocode

```
START
    INPUT attended, total
    IF total <= 0 OR attended < 0 OR attended > total THEN
        DISPLAY "Invalid attendance data"     ELSE
percentage = attended / total * 100         needed = 0
WHILE (attended + needed) / (total + needed) < 0.75
needed = needed + 1          END WHILE
        DISPLAY percentage, needed
    END IF
STOP
```

6. Vertical Flowchart



↓

STOP

7. Python Source Code

```
attended = int(input("Classes attended: ")) total
= int(input("Total classes: "))

if total <= 0 or attended < 0 or attended > total:
    print("Invalid attendance data") else:
    percentage = attended / total * 100     needed = 0
while (attended + needed) / (total + needed) < 0.75:
needed += 1     print("Current Attendance:",
round(percentage, 2), "%")     print("More classes to
attend:", needed)
```

8. Test Cases

Test Case	Input	Expected Result
1	60 attended out of 80	75%; needed 0
2	50 attended out of 80	62.5%; needed 40
3	8 attended out of 10	80%; needed 0

9. Sample Output

```
Classes attended: 50
Total classes: 80
Current Attendance: 62.5 %
More classes to attend: 40
```

10. Conclusion

The program uses a while loop to calculate the minimum classes needed to reach the attendance threshold.

CASE STUDY 7 – ATM TRANSACTION SEQUENCE SIMULATION

1. Problem Statement

Simulate deposits and withdrawals, reject invalid transactions, and display the final balance.

2. Objectives

1. Process transaction tuples.
2. Update account balance.
3. Reject invalid or unaffordable withdrawals.

3. Assumptions

• Opening balance is ■1,000.

• Each transaction is a type and amount tuple.

• Transaction amounts must be positive.

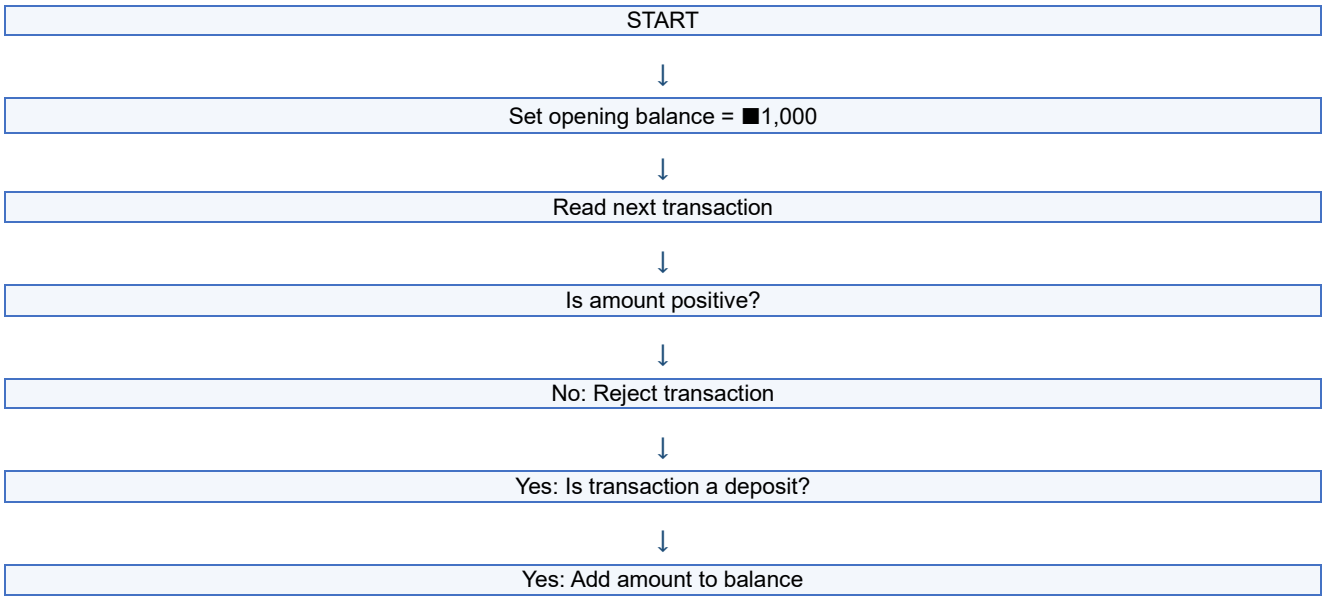
4. Algorithm

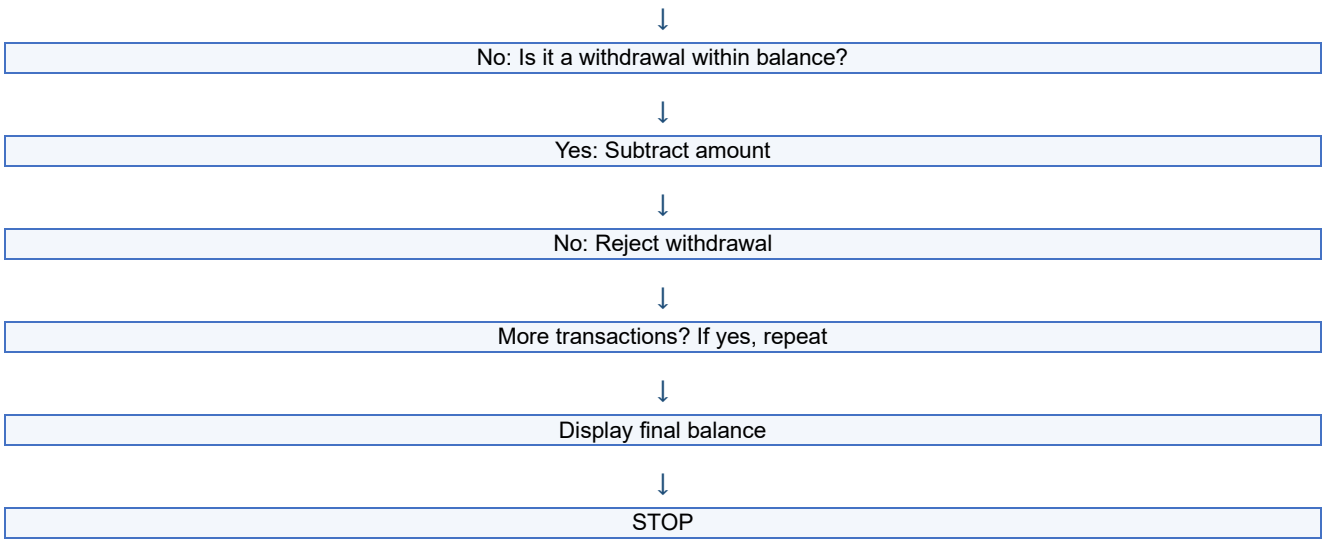
1. Set opening balance.
2. Process each transaction.
3. Add deposits to balance.
4. Allow withdrawals only when sufficient funds are available.
5. Display transaction status and final balance.

5. Pseudocode

```
START
    SET balance = 1000
    FOR each transaction (type, amount)
        IF amount <= 0 THEN
            DISPLAY "Invalid amount"         ELSE IF type =
"deposit" THEN             balance = balance + amount
ELSE IF type = "withdraw" AND amount <= balance THEN
balance = balance - amount         ELSE IF type = "withdraw"
THEN
            DISPLAY "Insufficient balance"
        ELSE
            DISPLAY "Invalid transaction type"
        END IF
    END FOR
DISPLAY balance
STOP
```

6. Vertical Flowchart





7. Python Source Code

```
balance = 1000
transactions = [
("deposit", 500),
    ("withdraw", 300),
    ("withdraw", 1500),
    ("deposit", 200)
]

for kind, amount in transactions:
if amount <= 0:
        print("Invalid amount:", amount)     elif
kind == "deposit":         balance += amount
print("Deposit successful:", amount)     elif kind
== "withdraw" and amount <= balance:
balance -= amount         print("Withdrawal
successful:", amount)     elif kind == "withdraw":
        print("Withdrawal rejected:", amount)
else:         print("Invalid transaction
type")

print("Final Balance:", balance)
```

8. Test Cases

Test Case	Input	Expected Result
1	Deposit ■500	Balance ■1,500
2	Withdraw ■300	Balance ■1,200
3	Withdraw ■1,500	Rejected: insufficient balance
4	Deposit ■200 after above transactions	Final balance ■1,400

9. Sample Output

```
Deposit successful: 500
Withdrawal successful: 300
Withdrawal rejected: 1500
Deposit successful: 200
Final Balance: 1400
```

10. Conclusion

The simulation demonstrates transaction processing with tuples, loops, conditions, and balance updates.

CASE STUDY 8 – WAREHOUSE INVENTORY RECONCILIATION

1. Problem Statement

Compare expected closing stock with recorded closing stock and identify discrepancies.

2. Objectives

1. Calculate expected closing stock.
2. Compare expected and recorded quantities.
3. Report mismatches.

3. Assumptions

• Expected stock = opening + incoming − dispatched.

• Stock quantities are whole numbers.

• A zero difference means records match.

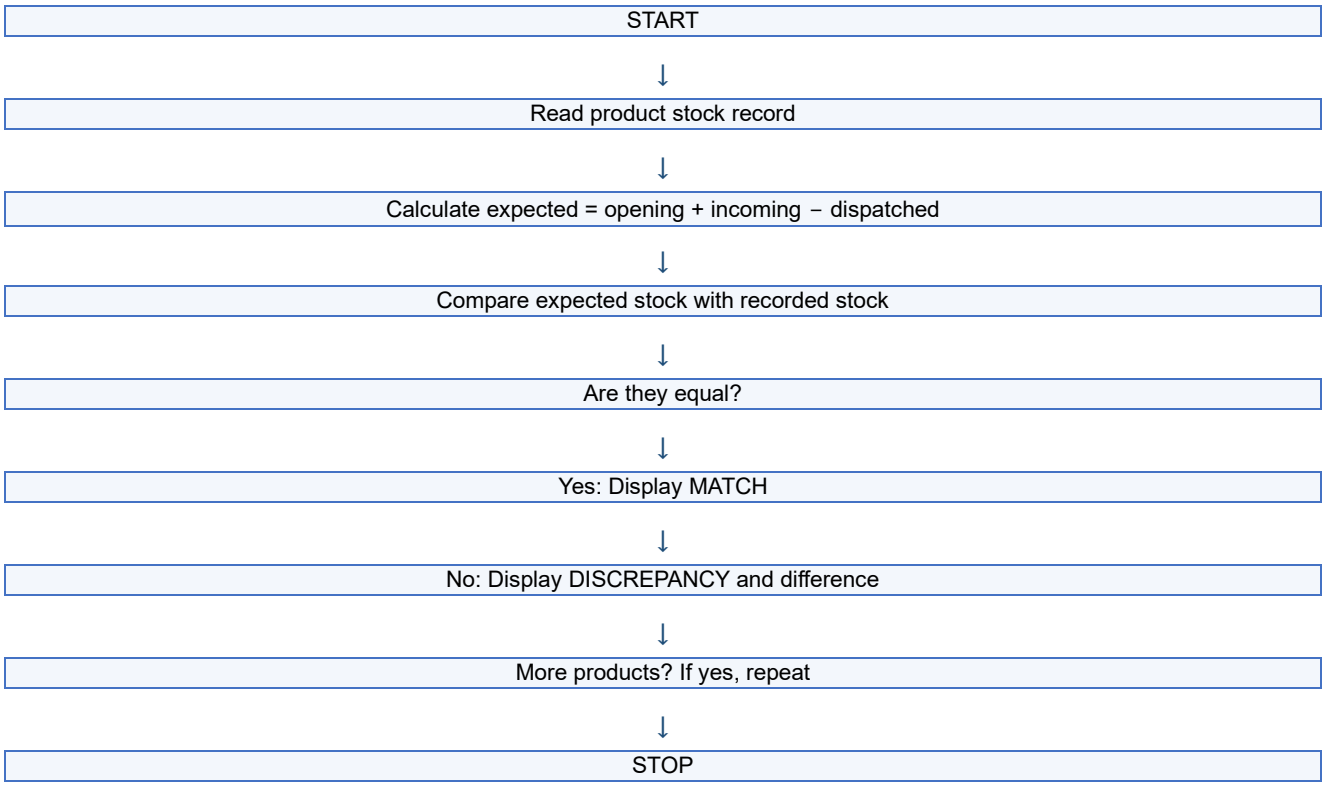
4. Algorithm

1. Store stock records as tuples.
2. Calculate expected closing stock.
3. Compare expected and recorded values.
4. Display match or discrepancy.

5. Pseudocode

```
START     FOR each product record         expected
= opening + incoming - dispatched
difference = recorded - expected         IF
difference = 0 THEN
            DISPLAY product, "MATCH"
        ELSE
            DISPLAY product, "DISCREPANCY", difference
        END IF
    END FOR
STOP
```

6. Vertical Flowchart



7. Python Source Code

```
stock = [     ("Pens", 100, 50,
80, 70),
    ("Books", 60, 20, 30, 55),
    ("Files", 40, 10, 15, 35)
] # (Product, opening, incoming, dispatched, recorded closing)
```

```
for name, opening, incoming, dispatched, recorded in stock:
    expected = opening + incoming - dispatched
difference = recorded - expected     if
difference == 0:
        print(name, "- MATCH:", expected)
else:
        print(name, "- DISCREPANCY:", difference,
"| Expected:", expected, "| Recorded:", recorded)
```

8. Test Cases

Test Case	Input	Expected Result
1	Pens: opening 100, incoming 50, dispatched 80, closing 70	MATCH
2	Books: opening 60, incoming 20, dispatched 30, closing 55	Discrepancy +5
3	Files: opening 40, incoming 10, dispatched 15, closing 35	MATCH

9. Sample Output

```
Pens - MATCH: 70
Books - DISCREPANCY: 5 | Expected: 50 | Recorded: 55
Files - MATCH: 35
```

10. Conclusion

The program reconciles inventory using tuples, arithmetic calculations, loops, and conditions.

CASE STUDY 9 – STUDENT MARKS MATRIX ANALYSIS

1. Problem Statement

Use a nested list to calculate student totals and averages, then identify the top performer.

2. Objectives

1. Store marks in a matrix.

2. Calculate each student's total and average.

3. Identify the highest total.

3. Assumptions

• Each row represents a student.

• Each student has marks in three subjects.

• Names are stored in a separate list.

4. Algorithm

1. Store student names and marks matrix. 2.

Calculate each row's total and average.

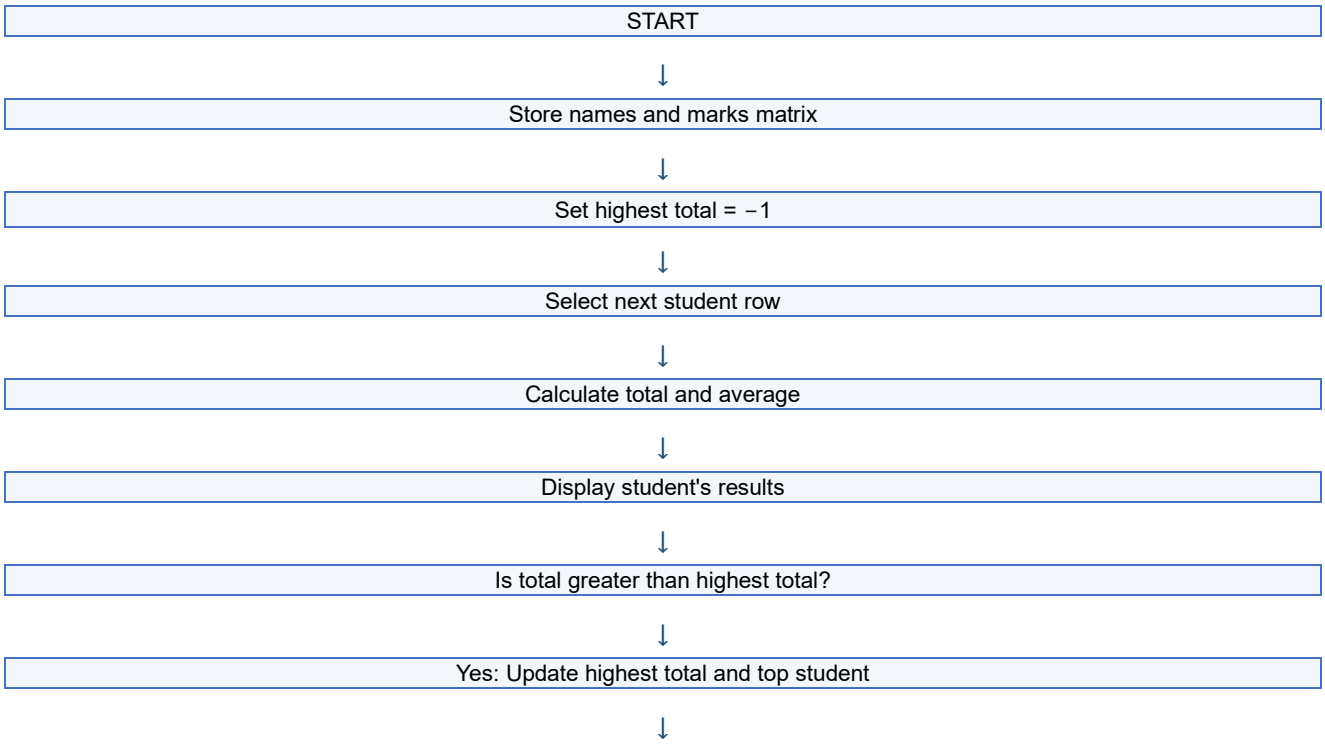
3. Track the highest total and student name.

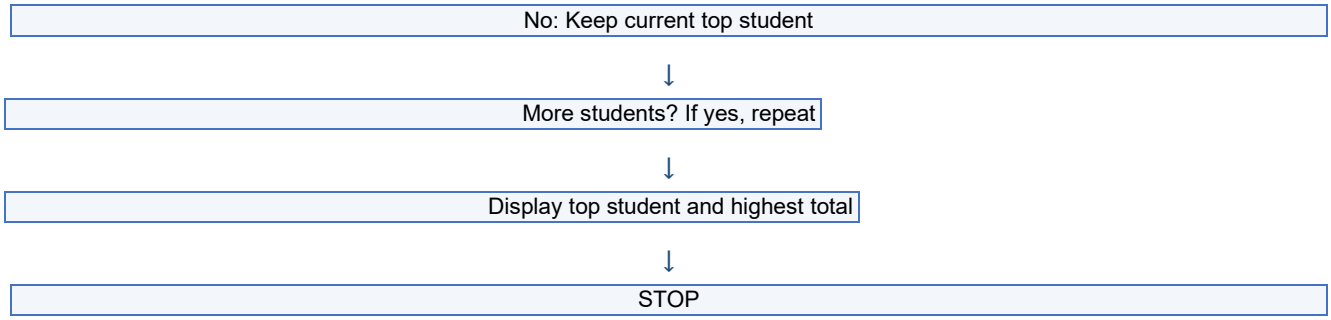
4. Display all results.

5. Pseudocode

```
START
    STORE student names and marks matrix
    SET highest_total = -1
    SET top_student = empty     FOR each
student index i         total = sum of marks
in row i          average = total / number of
subjects          DISPLAY student name, total,
average         IF total > highest_total THEN
highest_total = total             top_student
= student name         END IF
    END FOR
    DISPLAY top_student and highest_total
STOP
```

6. Vertical Flowchart





7. Python Source Code

```
names = ["Anil", "Bhavya", "Charan"]
marks = [     [80, 75, 90],
    [70, 85, 78],
    [92, 88, 95]
]

top_name = ""
highest = -1

for i in range(len(names)):     total = sum(marks[i])     average =
total / len(marks[i])     print(names[i], "Total:", total,
"Average:", round(average, 2))     if total > highest:
highest = total         top_name = names[i]

print("Top Student:", top_name) print("Highest
Total:", highest)
```

8. Test Cases

Test Case	Input	Expected Result
1	Anil: 80,75,90	Total 245; average 81.67
2	Bhavya: 70,85,78	Total 233; average 77.67
3	Charan: 92,88,95	Top student; total 275

9. Sample Output

```
Anil Total: 245 Average: 81.67
Bhavya Total: 233 Average: 77.67
Charan Total: 275 Average: 91.67
Top Student: Charan
Highest Total: 275
```

10. Conclusion

The program uses a nested list and loops to calculate results and identify the top student.

CASE STUDY 10 – TUPLE-BASED PRODUCT CATALOGUE AUDIT

1. Problem Statement

Calculate product inventory values and identify products with stock below the reorder level.

2. Objectives

1. Store product data in tuples.
2. Calculate inventory value.
3. Flag products needing replenishment.

3. Assumptions

• Each tuple contains product ID, price, and quantity.

• Reorder if quantity is below 10.

• Inventory value = price × quantity.

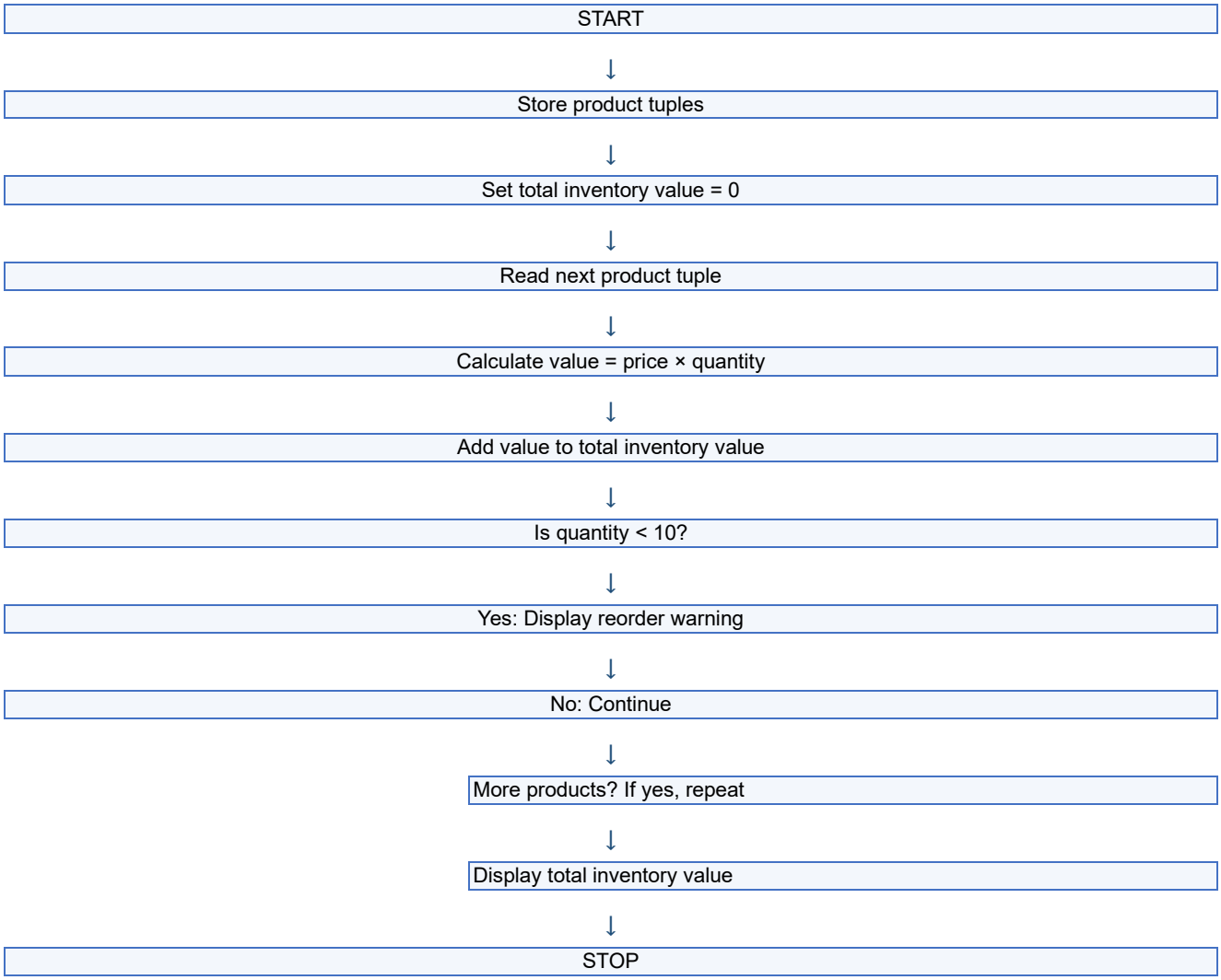
4. Algorithm

1. Store product records as tuples.
2. Calculate each product's inventory value.
3. Add each value to the overall total.
4. Flag stock quantities below 10.
5. Display product details and total inventory value.

5. Pseudocode

```
START
    SET total_value = 0     FOR each product
(product_id, price, quantity)          value = price
* quantity          total_value = total_value +
value          DISPLAY product_id, value, quantity
        IF quantity < 10 THEN
            DISPLAY "Reorder required"
        END IF
    END FOR
    DISPLAY total_value
STOP
```

6. Vertical Flowchart



7. Python Source Code

```
products = [
("P101", 25, 20),
    ("P102", 100, 5),
    ("P103", 40, 8),
    ("P104", 60, 15)
] # (Product ID, price,
quantity)
```

```
total_value = 0 print("Product
Catalogue Audit")

for product_id, price, quantity in products:
    value = price * quantity        total_value += value
print(product_id, "Value:", value, "Stock:", quantity)    if
quantity < 10:         print("  Reorder required")

print("Total Inventory Value:", total_value)
```

8. Test Cases

Test Case	Input	Expected Result
1	P101: price 25, qty 20	Value 500; no reorder
2	P102: price 100, qty 5	Value 500; reorder
3	P103: price 40, qty 8	Value 320; reorder
4	P104: price 60, qty 15	Value 900; no reorder

9. Sample Output

```
Product Catalogue Audit
P101 Value: 500 Stock: 20
P102 Value: 500 Stock: 5
  Reorder required
P103 Value: 320 Stock: 8
  Reorder required
P104 Value: 900 Stock: 15
Total Inventory Value: 2220
```

10. Conclusion

The program uses tuples, loops, and conditions to audit stock and calculate inventory value.